Name: Parag Manoj Mali
Student ID: 25201859
Github : https://github.com/maliparag11/Distributed-System-Lab04

COMP41720 Distributed Systems
LAB 4: Building a Microservices-Style Application

---

1. INTRODUCTION

The purpose of this lab was to design, implement, containerize, and deploy a simple distributed microservices application while demonstrating architectural reasoning and understanding of trade-offs. I implemented a URL Shortening system composed of two microservices:

- Shortener-Service (generates short codes)
- Redirect-Service (resolves codes to long URLs)

The system was implemented in Java 21 using Spring Boot 3.5, containerized using Docker, and deployed on Minikube Kubernetes. The services communicate via synchronous REST, and internal service discovery is handled using Kubernetes DNS.

This report explains service boundaries, inter-service communication, deployment architecture, key trade-offs, and rationale behind the design.

2. SYSTEM DESIGN & SETUP

2.1. ARCHITECTURE OVERVIEW

The URL Shortener consists of:

- Shortener-Service
  - Accepts long URLs
  - Generates random or custom short codes
  - Registers them with Redirect-Service via REST
- Redirect-Service
  - Stores code → long URL mappings
  - Resolves incoming short codes

Both services run independently and communicate over HTTP using Kubernetes networking.

2.2.ARCHITECTURE DIAGRAM

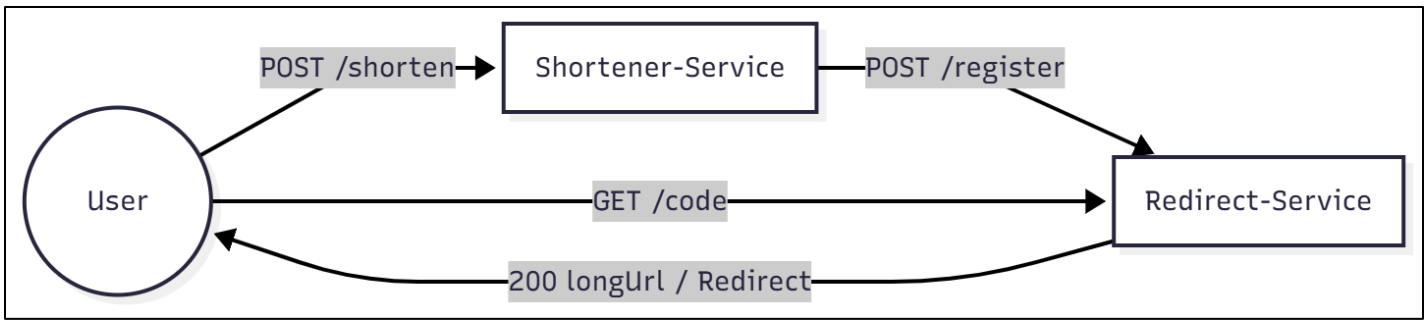

Fig 1 : Architecture Diagram

2.3.CONTAINERIZATION

Each Service has a minimal Dockerfile:

shortener-service:

```
FROM eclipse-temurin:21-jdk
WORKDIR /app

COPY target/shortener-service-0.0.1-SNAPSHOT.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

redirect-service:

```
FROM eclipse-temurin:21-jdk
WORKDIR /app

COPY target/redirect-service-0.0.1-SNAPSHOT.jar app.jar

EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Both containers were built inside Minikube's Docker environment using:

minikube docker-env

docker build -t shortener-service:1.0 .

docker build -t redirect-service:1.0 .

2.4.KUBERNETES DEPLOYMENT

Each service includes a Deployment + Service YML.

- shortener-service Deployment

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shortener-service
spec:
  replicas: 1
  selector:
    matchLabels:
      app: shortener-service
  template:
    metadata:
      labels:
        app: shortener-service
    spec:
      containers:
        - name: shortener-service
          image: shortener-service:1.0
          imagePullPolicy: Never
          ports:
            - containerPort: 8080
          env:
            - name: REDIRECT_BASE_URL
              value: "http://redirect-service:8080"
---
apiVersion: v1
kind: Service
metadata:
  name: shortener-service
spec:
  selector:
    app: shortener-service
  ports:
    - port: 9090
      targetPort: 8080
      nodePort: 30080
  type: NodePort

```

- redirect-service Deployment:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redirect-service
```

```yaml
spec:
  replicas: 1
  selector:
    matchLabels:
      app: redirect-service
  template:
    metadata:
      labels:
        app: redirect-service
    spec:
      containers:
        - name: redirect-service
          image: redirect-service:1.0
          imagePullPolicy: Never
          ports:
            - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: redirect-service
spec:
  selector:
    app: redirect-service
  ports:
    - port: 8080
      targetPort: 8080
  type: NodePort

```

2.5.SETUP & RUN INSTRUCTIONS

1. Start Minikube

minikube start

```
PS D:\UCD_CSNL_2025-26\DS_Labs\ParagManojMali_DistributedSystem_Lab04> minikube start
🤹  Updating the running docker "minikube" container ...
❗  Failing to connect to https://registry.k8s.io/ from inside the minikube container
💡  To pull new external images, you may need to configure a proxy: https://minikube.sigs.k8s.io/docs/reference/networking/proxy/
🐳  Preparing Kubernetes v1.34.0 on Docker 28.4.0 ...
🔎  Verifying Kubernetes components...
    ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌟  Enabled addons: storage-provisioner, default-storageclass
🏄  Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
```

2. Build JAR's

cd services\shortener-service
mvn clean package -DskipTests

```
PS D:\UCD_CSNL_2025-26\DS_Labs\ParagManojMali_DistributedSystem_Lab04\services\shortener-service> mvn clean packag
e -DskipTests
[INFO] The original artifact has been renamed to D:\UCD_CSNL_2025-26\DS_Labs\ParagManojMali_DistributedSystem_Lab0
4\services\shortener-service\target\shortener-service-0.0.1-SNAPSHOT.jar.original
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  8.929 s
[INFO] Finished at: 2025-11-16T12:20:56Z
[INFO] ------------------------------------------------------------------------
PS D:\UCD_CSNL_2025-26\DS_Labs\ParagManojMali_DistributedSystem_Lab04\services\shortener-service>
```

cd services\redirect-service
mvn clean package -DskipTests

```
PS D:\UCD_CSNL_2025-26\DS_Labs\ParagManojMali_DistributedSystem_Lab04> cd services/redirect-service
PS D:\UCD_CSNL_2025-26\DS_Labs\ParagManojMali_DistributedSystem_Lab04\services\redirect-service> mvn clean package
 -DskipTests
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  20.086 s
[INFO] Finished at: 2025-11-16T12:08:56Z
[INFO] ------------------------------------------------------------------------
```

3. Build Docker images inside Minikube

& minikube -p minikube docker-env --shell powershell | Invoke-Expression

```
PS D:\UCD_CSNL_2025-26\DS_Labs\ParagManojMali_DistributedSystem_Lab04> & minikube -p minikube docker-env --shell p
owershell | Invoke-Expression
```

docker build -t redirect-service:1.0 .

```
PS D:\UCD_CSNL_2025-26\DS_Labs\ParagManojMali_DistributedSystem_Lab04\services\redirect-service> docker build -t r
edirect-service:1.0 .
 => CACHED [2/3] WORKDIR /app                                                                    0.0s
 => [3/3] COPY target/redirect-service-0.0.1-SNAPSHOT.jar app.jar                                1.6s
 => exporting to image                                                                           0.2s
 => => exporting layers                                                                          0.2s
 => => writing image sha256:83ba5846adf8aa3b00c0e7d284e14bb7edeabd5c32c901e619e5a03a694b9e2f     0.0s
 => => naming to docker.io/library/redirect-service:1.0                                          0.0s

View build details: docker-desktop://dashboard/build/default/default/29wko3b900eqqoxacxkwh2uk9
```

docker build -t shortener-service:1.0 .

```
PS D:\UCD_CSNL_2025-26\DS_Labs\ParagManojMali_DistributedSystem_Lab04\services\shortener-service> docker build -t
redirect-service:1.0 .
 => CACHED [2/3] WORKDIR /app                                                                    0.0s
 => [3/3] COPY target/shortener-service-0.0.1-SNAPSHOT.jar app.jar                               1.3s
 => exporting to image                                                                           0.2s
 => => exporting layers                                                                          0.1s
 => => writing image sha256:7fb7e178681524f6995443ab539b62d1ca1e018651c3623189040dbbe02c2943     0.0s
 => => naming to docker.io/library/redirect-service:1.0                                          0.0s

View build details: docker-desktop://dashboard/build/default/default/ya39kub6z0vn8pihea64ltmn8
```

4. Apply Kubernetes manifests

kubectl apply -f redirect-deployment.yml

```
PS D:\UCD_CSNL_2025-26\DS_Labs\ParagManojMali_DistributedSystem_Lab04\k8s> kubectl apply -f redirect-deployment.ym
l
deployment.apps/redirect-service configured
service/redirect-service unchanged
```

kubectl apply -f shortener-deployment.yml

```
PS D:\UCD_CSNL_2025-26\DS_Labs\ParagManojMali_DistributedSystem_Lab04\k8s> kubectl apply -f shortener-deployment.y
ml
deployment.apps/shortener-service unchanged
service/shortener-service unchanged
```

5. Access shortener-service

Shortener-service URL :

minikube service shortener-service –url

```
PS D:\UCD_CSNL_2025-26\DS_Labs\ParagManojMali_DistributedSystem_Lab04\k8s> minikube service shortener-service --url
http://127.0.0.1:65004
❗  Because you are using a Docker driver on windows, the terminal needs to be open to run it.
```

Redirect-service URL:

minikube service redirectr-service –url

```
PS D:\UCD_CSNL_2025-26\DS_Labs\ParagManojMali_DistributedSystem_Lab04> minikube service redirect-service --url
http://127.0.0.1:60499
❗  Because you are using a Docker driver on windows, the terminal needs to be open to run it.
```

6. Test endpoints

To test the POST request:

Invoke-WebRequest -Uri "http://127.0.0.1:65004/shorten" `

>> -Method POST `

>> -ContentType "application/json" `
>> -Body "{ ""longUrl"": ""https://google.com"" }"

```
PS D:\UCD_CSNL_2025-26\DS_Labs\ParagManojMali_DistributedSystem_Lab04> Invoke-WebRequest -Uri "http://127.0.0.1:6500
4/shorten" `
>>   -Method POST `
>>   -ContentType "application/json" `
>>   -Body "{ ""longUrl"": ""https://google.com"" }"


StatusCode        : 201
StatusDescription :
Content           : {"code":"ryXSQS","shortUrl":"http://redirect-service:8080/ryXSQS"}
RawContent        : HTTP/1.1 201
                    Transfer-Encoding: chunked
                    Keep-Alive: timeout=60
                    Connection: keep-alive
                    Content-Type: application/json
                    Date: Sun, 16 Nov 2025 14:13:18 GMT

                    {"code":"ryXSQS","shortUrl":"http://re...
Forms             : {}
Headers           : {[Transfer-Encoding, chunked], [Keep-Alive, timeout=60], [Connection, keep-alive],
                    [Content-Type, application/json]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 66
```

Here, we can see that code for long url is created : ryXSQS. Now testing the URL which will give the result of long URL : https://google.com

Invoke-WebRequest -Uri http://127.0.0.1:60499/ryXSQS

```
PS D:\UCD_CSNL_2025-26\DS_Labs\ParagManojMali_DistributedSystem_Lab04> Invoke-WebRequest -Uri http://127.0.0.1:60499
/ryXSQS

StatusCode        : 200
StatusDescription :
Content           : {"code":"ryXSQS","longUrl":"https://google.com"}
RawContent        : HTTP/1.1 200
                    Transfer-Encoding: chunked
                    Keep-Alive: timeout=60
                    Connection: keep-alive
                    Content-Type: application/json
                    Date: Sun, 16 Nov 2025 14:28:36 GMT

                    {"code":"ryXSQS","longUrl":"https://go...
Forms             : {}
Headers           : {[Transfer-Encoding, chunked], [Keep-Alive, timeout=60], [Connection, keep-alive],
                    [Content-Type, application/json]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 48
```

3. ARCHITECTURAL ANALYSIS & JUSTIFICATION

3.1. Microservice Granularity

Service boundaries were split based on:

- Different scalability needs
- Different failure domains
- Different performance profiles

3.2. Inter-Service Communication Pattern

Synchronous REST chosen.

Positive Trade-offs:

- Simple implementation
- Strong consistency

Negative Trade-offs:

- Tight coupling
- Cascading failure risk

3.3. Deployment Architecture Analysis

Kubernetes provides:

- DNS discovery
- Restart automation
- Horizontal scaling

Trade-offs:
Higher operational complexity

4. ARCHITECTURAL DECISION RECORDS (ADRS)

- ADR 1 – Choosing Java + Spring Boot
- ADR 2 – Choosing synchronous REST
- ADR 3 – Splitting services based on scalability needs

5. CONCLUSION

This lab reinforced the importance of architectural trade-offs, microservice boundaries, communication choices, and Kubernetes orchestration.